MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 4

Struktur Perulangan: for, while, dan do-while



Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

Modul ini melanjutkan materi seleksi pada Modul 4. Fokus praktikum adalah membuat program yang menjalankan instruksi secara berulang berdasarkan jumlah tertentu atau selama suatu kondisi masih terpenuhi.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	5
Judul	Struktur Perulangan: for, while, dan do-while
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, tracing, eksperimen, debugging, dan latihan mandiri
Prasyarat	Modul 1–4

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- memilih struktur perulangan yang sesuai;
- menulis perulangan yang memiliki kondisi berhenti;
- menggunakan variabel counter secara benar;
- menggunakan accumulator untuk menghitung jumlah atau total;
- membedakan pre-test loop dan post-test loop;
- menjelaskan perbedaan for, while, dan do-while;
- membuat perulangan naik dan turun;
- menggunakan sentinel untuk mengakhiri input;
- melakukan validasi hingga pengguna memasukkan nilai yang benar;
- menghindari infinite loop;
- menentukan jumlah iterasi melalui trace table;
- menguji kondisi awal, kondisi batas, dan kondisi berhenti.

II. Persiapan Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_05/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
    ├── latihan_04.c
    ├── tracing.c
    ├── eksperimen.c
    ├── debugging.c
    └── tugas_modul_05.c
```

III. Konsep Dasar

1. Konsep Perulangan

Perulangan atau looping adalah proses menjalankan satu atau beberapa pernyataan secara berulang.

Tanpa perulangan:

```
printf("1\n");
printf("2\n");
printf("3\n");
printf("4\n");
printf("5\n");
```

Dengan perulangan:

```
int i;

for (i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Kedua program menghasilkan keluaran yang sama, tetapi perulangan lebih ringkas, mudah diubah, mudah digunakan untuk jumlah data yang besar, dan mudah dikembangkan untuk pemrosesan data.

Istilah Penting

Iterasi

Satu kali pelaksanaan blok perulangan disebut satu iterasi.

Counter

Variabel yang digunakan untuk menghitung jumlah perulangan.

```
int i;
```

Inisialisasi

Pemberian nilai awal kepada counter.

```
i = 1;
```

Kondisi

Ekspresi yang menentukan apakah perulangan dilanjutkan.

```
i <= 5
```

Update

Perubahan nilai counter setelah satu iterasi.

```
i++
```

2. Komponen Dasar Perulangan

Sebagian besar perulangan memiliki tiga komponen utama: (1) inisialisasi, (2) kondisi perulangan, (3) perubahan nilai.

Contoh:

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
    i++;
}
```

Komponennya adalah:

Komponen	Pernyataan
Inisialisasi	int i = 1;
Kondisi	i <= 5
Proses	printf("%d\n", i);
Update	i++;

Apabila bagian update dihilangkan:

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
}
```

nilai i selalu 1. Kondisi selalu benar dan program mengalami infinite loop.

3. Struktur for

Struktur for digunakan ketika jumlah perulangan telah diketahui atau dapat ditentukan sejak awal.

Bentuk Umum

```
for (inisialisasi; kondisi; update)
{
    pernyataan;
}
```

Contoh:

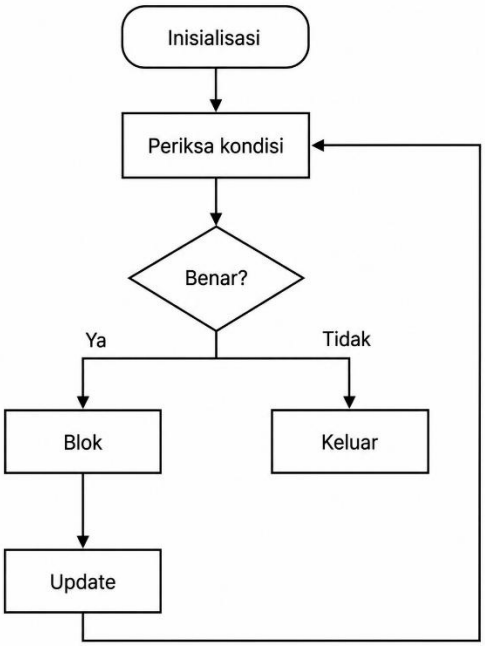
```
int i;

for (i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Keluaran:

```
1
2
3
4
5
```

Proses Eksekusi for



Berdasarkan gambar di atas, proses eksekusi perulangan for dimulai dari tahap **inisialisasi**, yang dijalankan satu kali pada awal perulangan. Setelah itu, program masuk ke tahap **pemeriksaan kondisi**. Jika kondisi bernilai **benar**, maka **blok perulangan** akan dijalankan, kemudian dilanjutkan dengan proses **update**. Setelah proses update selesai, alur kembali ke tahap **pemeriksaan kondisi**. Proses ini akan terus berulang selama kondisi masih bernilai benar. Sebaliknya, jika kondisi bernilai **salah**, maka perulangan akan berhenti dan program keluar dari struktur perulangan, sebagaimana ditunjukkan pada gambar.

Deklarasi Counter di dalam for

Dalam standar C modern, counter dapat dideklarasikan langsung:

```
for (int i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Variabel i hanya tersedia di dalam blok perulangan tersebut.

Alternatif:

```
int i;
for (i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Bentuk kedua digunakan apabila nilai counter masih diperlukan setelah perulangan.

4. Perulangan Naik dan Turun

Perulangan naik

```
int i;

for (i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
```

Keluaran:

```
1 2 3 4 5
```

Perulangan turun

```
int i;

for (i = 5; i >= 1; i--)
{
    printf("%d ", i);
}
```

Keluaran:

```
5 4 3 2 1
```

Perubahan lebih dari satu

```
int i;

for (i = 0; i <= 10; i += 2)
{
    printf("%d ", i);
}
```

Keluaran:

```
0 2 4 6 8 10
```

5. Counter dan Accumulator

Counter

Counter menghitung banyaknya kejadian.

```
int jumlah_lulus = 0;

if (nilai >= 60)
{
    jumlah_lulus++;
}
```

Accumulator

Accumulator menyimpan hasil penjumlahan secara bertahap.

```
double total_nilai = 0.0;

total_nilai += nilai;
```

Contoh program:

```
#include <stdio.h>

int main(void)
{
    int i;
    int nilai;
    int jumlah_lulus = 0;
    int total_nilai = 0;

    for (i = 1; i <= 3; i++)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", &nilai);

        total_nilai += nilai;

        if (nilai >= 60)
        {
            jumlah_lulus++;
        }
    }

    printf("Total nilai  : %d\n", total_nilai);
    printf("Jumlah lulus : %d\n", jumlah_lulus);

    return 0;
}
```

Perbedaan:

Variabel	Fungsi
i	menghitung iterasi
jumlah_lulus	menghitung mahasiswa yang lulus
total_nilai	menjumlahkan seluruh nilai

6. Menghitung Jumlah Iterasi

Perhatikan:

```
for (int i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Nilai counter:

```
1, 2, 3, 4, 5
```

Jumlah iterasi adalah 5.

Perhatikan contoh lain:

```
for (int i = 0; i < 5; i++)
{
    printf("%d\n", i);
}
```

Nilai counter:

```
0, 1, 2, 3, 4
```

Jumlah iterasi tetap 5.

Kesalahan dalam menentukan batas sering disebut off-by-one error.

7. Off-by-One Error

Off-by-one error terjadi ketika perulangan berjalan satu kali terlalu banyak atau satu kali terlalu sedikit.

Contoh:

```
for (int i = 1; i < 5; i++)
{
    printf("%d\n", i);
}
```

Keluaran:

```
1
2
3
4
```

Nilai 5 tidak ditampilkan karena kondisi menggunakan:

```
i < 5
```

Apabila ingin menampilkan 1 sampai 5:

```
i <= 5
```

Contoh lain:

```
for (int i = 0; i <= 5; i++)
```

menghasilkan 6 iterasi:

```
0, 1, 2, 3, 4, 5
```

Mahasiswa harus memperhatikan: nilai awal, operator relasional, nilai akhir, dan besar perubahan counter.

8. Struktur while

Struktur while digunakan ketika jumlah iterasi belum tentu diketahui sejak awal.

Bentuk Umum

```
while (kondisi)
{
    pernyataan;
}
```

Contoh:

```
int angka = 1;

while (angka <= 5)
{
    printf("%d\n", angka);
    angka++;
}
```

Keluaran:

```
1
2
3
4
5
```

while sebagai Pre-Test Loop

Pada while, kondisi diperiksa sebelum blok dijalankan.

```
int angka = 10;

while (angka <= 5)
{
    printf("%d\n", angka);
    angka++;
}
```

Karena kondisi awal salah, blok perulangan tidak pernah dijalankan.

Oleh karena itu, while disebut pre-test loop.

Kapan Menggunakan while?

Gunakan while ketika:

- jumlah iterasi tidak diketahui;
- perulangan bergantung pada kondisi;
- proses dilakukan selama input masih valid;
- proses dilakukan sampai target tercapai;
- program menunggu nilai sentinel;
- program mengulangi proses selama pengguna memilih lanjut.

9. Perulangan Berbasis Sentinel

Sentinel adalah nilai khusus yang digunakan untuk menghentikan perulangan.

Contoh:

```
#include <stdio.h>

int main(void)
{
    int nilai;
    int total = 0;
    int jumlah_data = 0;

    printf("Masukkan nilai, gunakan -1 untuk berhenti: ");
    scanf("%d", &nilai);

    while (nilai != -1)
    {
        total += nilai;
        jumlah_data++;

        printf("Masukkan nilai, gunakan -1 untuk berhenti: ");
        scanf("%d", &nilai);
    }

    printf("Jumlah data : %d\n", jumlah_data);
    printf("Total nilai : %d\n", total);

    return 0;
}
```

Nilai -1 tidak ikut diproses karena hanya digunakan sebagai tanda berhenti.

Pemilihan Sentinel

Sentinel harus dipilih dari nilai yang tidak termasuk data valid. Apabila data valid berada pada rentang 0–100, nilai -1 dapat digunakan sebagai sentinel. Sentinel yang dipilih harus mudah dikenali, tidak bertabrakan dengan data normal, dijelaskan kepada pengguna.

10. Menghindari Pembagian dengan Nol

Perhatikan program rata-rata menggunakan sentinel.

```
double rata_rata;
rata_rata = (double) total / jumlah_data;
```

Apabila pengguna langsung memasukkan -1, maka:

```
jumlah_data = 0
```

Program tidak boleh melakukan pembagian dengan nol.

Gunakan validasi:

```
if (jumlah_data > 0)
{
    double rata_rata = (double) total / jumlah_data;
    printf("Rata-rata: %.2f\n", rata_rata);
}
else
{
    printf("Tidak ada data yang diproses.\n");
}
```

11. Struktur do-while

Struktur do-while digunakan apabila blok harus dijalankan minimal satu kali.

Bentuk Umum

```
do
{
    pernyataan;
}
while (kondisi);
```

Perhatikan titik koma setelah while (kondisi);

Contoh:

```
int angka = 1;
do
{
    printf("%d\n", angka);
    angka++;
}
while (angka <= 5);
```

do-while sebagai Post-Test Loop

Pada do-while:

- blok dijalankan;
- kondisi diperiksa;
- apabila kondisi benar, blok diulangi.

Contoh:

```
int angka = 10;

do
{
    printf("%d\n", angka);
    angka++;
}
while (angka <= 5);
```

Keluaran:

```
10
```

Meskipun kondisi salah, blok tetap dijalankan satu kali.

Karena kondisi diperiksa setelah blok dijalankan, do-while disebut post-test loop.

12. Validasi Input Menggunakan do-while

Contoh meminta nilai 0–100:

```
#include <stdio.h>

int main(void)
{
    double nilai;

    do
    {
        printf("Masukkan nilai 0-100: ");
        scanf("%lf", &nilai);

        if ((nilai < 0.0) || (nilai > 100.0))
        {
            printf("Nilai tidak valid. Silakan ulangi.\n");
        }
    }
    while ((nilai < 0.0) || (nilai > 100.0));

    printf("Nilai yang diterima: %.2f\n", nilai);

    return 0;
}
```

Blok input harus dijalankan minimal satu kali sehingga do-while sesuai untuk kasus ini.

13. Perbedaan for, while, dan do-while

Aspek	for	while	do-while
Kondisi diperiksa	Sebelum blok	Sebelum blok	Setelah blok
Minimal eksekusi	0 kali	0 kali	1 kali
Jumlah iterasi	Umumnya diketahui	Umumnya tidak diketahui	Umumnya tidak diketahui
Inisialisasi dan update	Terlihat pada kepala loop	Ditulis terpisah	Ditulis terpisah
Cocok untuk	Pengulangan berdasarkan jumlah	Kondisi atau sentinel	Menu dan validasi input

Pedoman pemilihan

Gunakan for ketika: “Ulangi sebanyak n kali.”

Gunakan while ketika: “Ulangi selama kondisi masih benar.”

Gunakan do-while ketika: “Lakukan proses minimal sekali, kemudian ulangi apabila diperlukan.”

Ketiga struktur dapat menghasilkan perilaku yang sama pada banyak kasus, tetapi pilih bentuk yang paling mudah dibaca.

14. Pernyataan break

break menghentikan perulangan sebelum kondisi normalnya berakhir.

Contoh:

```
int i;

for (i = 1; i <= 10; i++)
{
    if (i == 6)
    {
        break;
    }

    printf("%d ", i);
}
```

Keluaran:

```
1 2 3 4 5
```

Ketika i bernilai 6, perulangan dihentikan.

Penggunaan yang sesuai

break dapat digunakan ketika: 1) data yang dicari telah ditemukan; 2) pengguna memilih keluar; dan 3) terjadi kondisi yang membuat proses tidak dapat dilanjutkan.

break tidak sebaiknya digunakan untuk menutupi kondisi perulangan yang dirancang dengan buruk.

15. Pernyataan continue

continue melewati sisa pernyataan pada iterasi saat ini dan melanjutkan ke iterasi berikutnya.

Contoh:

```
int i;

for (i = 1; i <= 5; i++)
{
    if (i == 3)
    {
        continue;
    }

    printf("%d ", i);
}
```

Keluaran:

```
1 2 4 5
```

Ketika i bernilai 3, pernyataan printf() dilewati.

Gunakan secara terbatas

Penggunaan continue yang berlebihan dapat membuat alur program sulit ditelusuri. Sering kali kondisi positif lebih mudah dibaca.

Menggunakan continue:

```
if (nilai < 0)
{
    continue;
}

total += nilai;
```

Alternatif:

```
if (nilai >= 0)
{
    total += nilai;
}
```

16. Infinite Loop

Infinite loop adalah perulangan yang tidak pernah berhenti.

Update tidak dilakukan

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
}
```

Nilai i tidak pernah berubah.

Update bergerak ke arah yang salah

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
    i--;
}
```

Nilai i semakin kecil sehingga kondisi tetap benar.

Kondisi selalu benar

```
while (1)
{
    printf("Program terus berjalan.\n");
}
```

Bentuk ini kadang digunakan dengan break, tetapi belum disarankan untuk mahasiswa pemula kecuali tujuannya jelas.

Cara menghentikan program

Apabila terjadi infinite loop:

- gunakan tombol stop pada IDE;
- pada terminal, gunakan Ctrl + C;
- periksa inisialisasi, kondisi, dan update.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Menampilkan Deret Menggunakan for

Buat berkas: latihan_01.c. Program:

```
#include <stdio.h>

int main(void)
{
    int batas;
    int i;

    printf("Masukkan batas akhir: ");
    scanf("%d", &batas);

    if (batas < 1)
    {
        printf("Batas harus minimal 1.\n");
        return 1;
    }

    printf("Deret bilangan:\n");

    for (i = 1; i <= batas; i++)
    {
        printf("%d ", i);
    }

    printf("\n");

    return 0;
}
```

Contoh

Input: 5

Keluaran:

```
1 2 3 4 5
```

Pertanyaan analisis

1. Apa nilai awal counter?
2. Apa kondisi perulangannya?
3. Apa bentuk update counter?
4. Berapa jumlah iterasi untuk input 5?
5. Apa hasil apabila kondisi diubah menjadi i < batas?
6. Apa hasil apabila nilai awal diubah menjadi 0?
7. Apa hasil apabila update diubah menjadi i += 2?
8. Mengapa input kurang dari 1 ditolak?

2. Praktikum Terbimbing 2 — Menghitung Jumlah dan Rata-Rata

Buat berkas: latihan_02.c. Program menerima sejumlah bilangan sesuai banyak data yang ditentukan pengguna.

```
#include <stdio.h>
int main(void)
{
    int banyak_data;
    int i;
    double nilai;
    double total = 0.0;
    double rata_rata;

    printf("Masukkan banyak data: ");
    scanf("%d", &banyak_data);

    if (banyak_data <= 0)
    {
        printf("Banyak data harus lebih besar dari nol.\n");
        return 1;
    }

    for (i = 1; i <= banyak_data; i++)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%lf", &nilai);

        total += nilai;
    }

    rata_rata = total / banyak_data;

    printf("\nHASIL\n");
    printf("Banyak data : %d\n", banyak_data);
    printf("Total       : %.2f\n", total);
    printf("Rata-rata   : %.2f\n", rata_rata);

    return 0;
}
```

Contoh

```
Banyak data: 4
Nilai ke-1: 70
Nilai ke-2: 80
Nilai ke-3: 90
Nilai ke-4: 60
```

Keluaran:

```
Banyak data : 4
Total       : 300.00
Rata-rata   : 75.00
```

Pertanyaan analisis

1. Mengapa total diinisialisasi dengan 0.0?
2. Apa fungsi total += nilai?
3. Mengapa banyak_data tidak boleh nol?
4. Berapa kali perulangan berjalan apabila banyak data adalah 4?
5. Apa yang terjadi apabila nilai total tidak diinisialisasi?
6. Apakah rata-rata membutuhkan casting?
7. Mengapa dalam program ini casting tidak wajib?

3. Praktikum Terbimbing 3 — Input Berbasis Sentinel Menggunakan while

Buat berkas: latihan_03.c. Program menerima nilai 0–100. Input berhenti ketika pengguna memasukkan -1.

```
#include <stdio.h>

int main(void)
{
    double nilai;
    double total = 0.0;
    double rata_rata;
    double nilai_tertinggi = 0.0;
    double nilai_terendah = 0.0;

    int jumlah_data = 0;
    int jumlah_lulus = 0;

    printf("Masukkan nilai 0-100, gunakan -1 untuk berhenti.\n");
    printf("Nilai: ");
    scanf("%lf", &nilai);

    while (nilai != -1.0)
    {
        if ((nilai < 0.0) || (nilai > 100.0))
        {
            printf("Nilai tidak valid dan tidak diproses.\n");
        }
        else
```

```
        {
            if (jumlah_data == 0)
            {
                nilai_tertinggi = nilai;
                nilai_terendah = nilai;
            }
            else
            {
                if (nilai > nilai_tertinggi)
                {
                    nilai_tertinggi = nilai;
                }

                if (nilai < nilai_terendah)
                {
                    nilai_terendah = nilai;
                }
            }

            total += nilai;
            jumlah_data++;

            if (nilai >= 60.0)
            {
                jumlah_lulus++;
            }
        }

        printf("Nilai: ");
        scanf("%lf", &nilai);
    }

    if (jumlah_data > 0)
    {
        rata_rata = total / jumlah_data;

        printf("\nHASIL REKAP\n");
        printf("Jumlah data    : %d\n", jumlah_data);
        printf("Total nilai    : %.2f\n", total);
        printf("Rata-rata      : %.2f\n", rata_rata);
        printf("Nilai tertinggi: %.2f\n", nilai_tertinggi);
        printf("Nilai terendah : %.2f\n", nilai_terendah);
        printf("Jumlah lulus   : %d\n", jumlah_lulus);
    }
    else
    {
        printf("Tidak ada data valid yang diproses.\n");
    }

    return 0;
}
```

Pertanyaan analisis

1. Mengapa input pertama dilakukan sebelum while?
2. Mengapa input kembali dilakukan di akhir blok?
3. Apakah nilai -1 ikut dihitung?
4. Mengapa jumlah_data diperiksa sebelum menghitung rata-rata?
5. Mengapa nilai tertinggi dan terendah diinisialisasi dari data pertama?
6. Apa yang terjadi apabila pengguna langsung memasukkan -1?
7. Apa yang terjadi apabila pengguna memasukkan 120?
8. Apakah 120 menambah jumlah_data?
9. Berapa jumlah lulus jika data adalah 50, 60, 70, dan 80?

4. Praktikum Terbimbing 4 — Validasi Input Menggunakan do-while

Buat berkas: latihan_04.c. Program meminta pengguna memasukkan jumlah barang antara 1 dan 100.

```
#include <stdio.h>

int main(void)
{
    int jumlah_barang;

    do
    {
        printf("Masukkan jumlah barang 1-100: ");
        scanf("%d", &jumlah_barang);

        if ((jumlah_barang < 1) || (jumlah_barang > 100))
        {
            printf("Input tidak valid. Silakan ulangi.\n");
        }
    }
    while ((jumlah_barang < 1) || (jumlah_barang > 100));

    printf("Jumlah barang yang diterima: %d\n", jumlah_barang);

    return 0;
}
```

Contoh interaksi

```
Masukkan jumlah barang 1-100: 0
Input tidak valid. Silakan ulangi.
Masukkan jumlah barang 1-100: 120
Input tidak valid. Silakan ulangi.
Masukkan jumlah barang 1-100: 25
Jumlah barang yang diterima: 25
```

Pertanyaan analisis

1. Mengapa do-while tepat untuk kasus ini?
2. Berapa kali blok dijalankan minimal?
3. Apa kondisi agar perulangan terus berjalan?
4. Apa kondisi agar perulangan berhenti?
5. Mengapa terdapat titik koma setelah while?
6. Apa akibatnya apabila kondisi diubah menjadi &&?
7. Dapatkah program ini dibuat menggunakan while?

V. Eksperimen dan Tracing

1. Latihan Tracing

Buat berkas: tracing.c. Jangan langsung menjalankan program.

```
#include <stdio.h>

int main(void)
{
    int i;
    int total = 0;
    int jumlah_genap = 0;

    for (i = 1; i <= 6; i++)
    {
        total += i;

        if (i % 2 == 0)
        {
            jumlah_genap++;
        }
    }

    printf("i            = %d\n", i);
    printf("total        = %d\n", total);
    printf("jumlah genap = %d\n", jumlah_genap);

    return 0;
}
```

Lengkapi tabel berikut.

Iterasi	i	total sebelum	total sesudah	i % 2 == 0	jumlah_genap
1	1	0			
2	2				
3	3				
4	4				
5	5				
6	6				

Setelah tabel selesai:

- prediksi nilai akhir i;
- prediksi nilai total;
- prediksi nilai jumlah_genap;
- kompilasi dan jalankan program;
- bandingkan hasil aktual dengan hasil tracing.

Perhatikan bahwa setelah perulangan selesai, nilai i menjadi 7 karena kondisi i <= 6 baru dinyatakan salah saat i bernilai 7.

2. Eksperimen

Buat berkas: eksperimen.c. Gunakan program dasar:

```
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }

    printf("\n");

    return 0;
}
```

Lakukan eksperimen berikut satu per satu.

No.	Perubahan	Prediksi	Hasil Aktual
1	Ubah i <= 5 menjadi i < 5		
2	Ubah nilai awal menjadi i = 0		
3	Ubah update menjadi i += 2		
4	Ubah update menjadi i--		

5	Ubah kondisi menjadi i >= 1 dengan update i++		
6	Pindahkan printf() setelah perulangan		
7	Tambahkan break ketika i == 3		
8	Tambahkan continue ketika i == 3		
9	Ubah program menjadi while		
10	Ubah program menjadi do-while		

Untuk setiap eksperimen:

- tuliskan prediksi;
- periksa apakah perubahan aman dijalankan;
- kompilasi program;
- jalankan apabila tidak menyebabkan infinite loop;
- catat hasil;
- jelaskan perbedaannya;
- kembalikan program ke kondisi awal.

Eksperimen nomor 4 dan 5 berpotensi menyebabkan infinite loop. Analisis terlebih dahulu sebelum menjalankannya.

VI. Debugging Challenge

1. Debugging Challenge

Program berikut dimaksudkan untuk menerima lima nilai, menghitung total, rata-rata, jumlah lulus, dan jumlah tidak lulus.

```
#include <stdio.h>

int main(void)
{
    int i;
    int nilai;
    int total;
    int jumlah_lulus = 0;
    int jumlah_tidak_lulus = 0;
    double rata_rata;

    for (i = 1; i < 5; i--)
    {
        printf("Masukkan nilai ke-%d: ", i);
        scanf("%d", nilai);

        if (nilai < 0 && nilai > 100)
        {
            printf("Nilai tidak valid.\n");
        }
```

```
        else
        {
            total = nilai;

            if (nilai >= 60);
            {
                jumlah_lulus++;
            }
            else
            {
                jumlah_tidak_lulus++;
            }
        }
    }

    rata_rata = total / 5;

    printf("Total              : %d\n", total);
    printf("Rata-rata          : %.2f\n", rata_rata);
    printf("Jumlah lulus       : %d\n", jumlah_lulus);
    printf("Jumlah tidak lulus : %d\n", jumlah_tidak_lulus);

    return 0;
}
```

Tugas

1. Salin program ke debugging.c.
2. Jangan langsung menjalankan program sebelum memeriksa risiko infinite loop.
3. Identifikasi kesalahan inisialisasi.
4. Identifikasi kesalahan kondisi dan update for.
5. Perbaiki penggunaan scanf().
6. Perbaiki kondisi validasi nilai.
7. Perbaiki proses accumulator.
8. Periksa titik koma setelah if.
9. Pastikan rata-rata menghasilkan nilai pecahan.
10. Pastikan program hanya memproses lima nilai valid.

Kesalahan yang Perlu Ditemukan

- total belum diinisialisasi;
- kondisi perulangan salah;
- update counter bergerak ke arah yang salah;
- scanf() tidak menggunakan &;
- validasi menggunakan operator logika yang salah;
- total = nilai seharusnya menambahkan nilai;
- terdapat titik koma setelah if;

- pembagian rata-rata menggunakan pembagian bilangan bulat;
- penanganan nilai tidak valid belum menjamin jumlah data valid tetap lima.

Tabel Debugging

No.	Bagian yang Salah	Jenis Kesalahan	Penyebab	Perbaikan
1				
2				
3				
4				
5				
6				
7				
8				

VII. Latihan Mandiri

1. Level 1 — Menampilkan Bilangan

Buat program yang menerima batas awal dan batas akhir, kemudian menampilkan seluruh bilangan di antara keduanya.

Contoh:

```
Masukkan batas awal : 3
Masukkan batas akhir: 8

3 4 5 6 7 8
```

Ketentuan:

- batas awal harus lebih kecil atau sama dengan batas akhir;
- gunakan for;
- tampilkan pesan kesalahan apabila batas tidak valid.

2. Level 2 — Menghitung Bilangan Genap dan Ganjil

Buat program yang menerima bilangan bulat positif n, kemudian menghitung jumlah dan total bilangan genap serta ganjil dari 1 sampai n.

Contoh:

```
Masukkan n: 6

Jumlah bilangan genap : 3
Jumlah bilangan ganjil: 3
Total bilangan genap  : 12
Total bilangan ganjil : 9
```

Gunakan: for, operator %, counter, accumulator.

3. Level 3 — Menghitung Faktorial

Buat program yang menghitung faktorial bilangan bulat non-negatif.

Definisi:

```
n! = n × (n − 1) × ... × 2 × 1
```

Contoh: 5! = 5 × 4 × 3 × 2 × 1 = 120

Ketentuan:

- nilai input minimal 0;
- 0! = 1;
- gunakan perulangan;
- gunakan tipe unsigned long long;
- batasi input maksimal 20 untuk mengurangi risiko overflow.

Contoh:

```
Masukkan bilangan 0-20: 5
5! = 120
```

4. Challenge — Membentuk Bilangan Terbalik

Buat program yang menerima satu bilangan bulat positif dan membalik susunan digitnya.

Contoh:

```
Masukkan bilangan: 5724
Bilangan terbalik: 4275
```

Petunjuk:

```
digit = bilangan % 10
hasil = hasil × 10 + digit
bilangan = bilangan / 10
```

Gunakan while.

Contoh tracing untuk 572:

Iterasi	Bilangan	Digit	Hasil
Awal	572	–	0
1	57	2	2
2	5	7	27
3	0	5	275

5. Jumlah Digit Bilangan

Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.

Sebuah aplikasi validasi kode ingin menghitung jumlah digit dari suatu bilangan (mirip perhitungan check-digit sederhana). Buat program C yang menerima satu bilangan bulat positif dan menghitung jumlah seluruh digitnya menggunakan while.

Format Input

Satu baris bilangan bulat positif.

Batasan

- $1 \leq$ bilangan ≤ 999999999.

Contoh 1

Input:

```
1234
```

Output:

```
=======================================
         JUMLAH DIGIT BILANGAN
=======================================
Bilangan       : 1234
Jumlah Digit   : 10
=======================================
```

Contoh 2

Input:

```
999
```

Output:

```
=======================================
         JUMLAH DIGIT BILANGAN
=======================================
Bilangan       : 999
Jumlah Digit   : 27
=======================================
```

Contoh 3

Input:

```
500000
```

Output:

```
=======================================
         JUMLAH DIGIT BILANGAN
=======================================
Bilangan       : 500000
Jumlah Digit   : 5
=======================================
```

Petunjuk Pendekatan

- gunakan variabel bertipe int untuk bilangan, sisa bilangan yang diproses, dan accumulator jumlah;
- gunakan while (sisa > 0) untuk mengambil digit satu per satu menggunakan sisa % 10, lalu perbarui sisa dengan sisa / 10 di setiap iterasi;
- tampilkan bilangan asli pada label “Bilangan”, bukan variabel sisa yang sudah berubah selama perulangan;

- label “Jumlah Digit” pada soal ini berarti jumlah (total) dari seluruh digit, bukan banyaknya digit — perhatikan Contoh 1: 1+2+3+4 = 10;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

Simpan sebagai latihan_bonus_05.c.

VIII. Pengujian Program

1. Pengujian Program Perulangan

Program perulangan harus diuji pada:

a. Iterasi minimum

Contoh: n = 1

b. Nilai batas

Contoh faktorial: n = 0, n = 20

c. Nilai tidak valid

Contoh: n = -1, n = 21

d. Jumlah iterasi lebih besar

Contoh: n = 100

e. Kondisi berhenti

Pastikan program benar-benar berhenti pada kondisi yang dirancang.

Contoh Tabel Pengujian Faktorial

No.	Input	Expected Output	Actual Output	Status
1	-1	Input tidak valid		
2	0	1		
3	1	1		
4	5	120		
5	10	3.628.800		
6	20	2.432.902.008.176.640.000		
7	21	Input tidak valid		

IX. Tugas Praktikum

1. Tugas Praktikum Modul 5 — Program Rekap Nilai Mahasiswa

Buat program untuk menerima dan mengolah sejumlah nilai mahasiswa.

1. Input

Program menerima jumlah mahasiswa dan nilai setiap mahasiswa.

Ketentuan:

- jumlah mahasiswa harus berada dalam rentang 1–100;
- nilai setiap mahasiswa harus berada dalam rentang 0–100;
- input yang tidak memenuhi rentang harus diminta kembali.

2. Proses

Program menghitung: total nilai, rata-rata nilai, nilai tertinggi, nilai terendah, jumlah mahasiswa lulus, jumlah mahasiswa tidak lulus, persentase kelulusan.

Mahasiswa dinyatakan lulus apabila nilai ≥ 60.

Persentase kelulusan:

```
persentase lulus =
jumlah lulus ÷ jumlah mahasiswa × 100%
```

3. Contoh Interaksi

```
================================================
              PROGRAM REKAP NILAI
================================================
Masukkan jumlah mahasiswa 1-100: 5

Nilai mahasiswa ke-1: 80
Nilai mahasiswa ke-2: 55
Nilai mahasiswa ke-3: 70
Nilai mahasiswa ke-4: 90
Nilai mahasiswa ke-5: 60

HASIL REKAP
------------------------------------------------
Jumlah mahasiswa     : 5
Total nilai          : 355.00
Rata-rata            : 71.00
Nilai tertinggi      : 90.00
Nilai terendah       : 55.00
Jumlah lulus         : 4
Jumlah tidak lulus   : 1
Persentase kelulusan : 80.00%
================================================
```

4. Struktur Perulangan yang Digunakan

Program wajib menggunakan:

- do-while untuk memvalidasi jumlah mahasiswa;
- for untuk menerima nilai mahasiswa;
- do-while untuk memvalidasi setiap nilai;
- counter untuk jumlah lulus dan tidak lulus;
- accumulator untuk total nilai.

5. Tahapan Algoritma

1. meminta jumlah mahasiswa;
2. mengulang input selama jumlah mahasiswa tidak valid;
3. menginisialisasi total dan counter;
4. melakukan perulangan sebanyak jumlah mahasiswa;

5. meminta nilai mahasiswa;
6. mengulang input nilai apabila berada di luar rentang 0–100;
7. menambahkan nilai ke total;
8. menentukan nilai tertinggi dan terendah;
9. menentukan status lulus;
10. menghitung rata-rata;
11. menghitung persentase kelulusan;
12. menampilkan hasil.

6. Analisis IPO

Mahasiswa wajib melengkapi tabel berikut.

Komponen	**Uraian**
Input	
Process	
Output	

7. Daftar Variabel

Mahasiswa wajib menjelaskan variabel yang digunakan.

Variabel	**Tipe Data**	**Nilai Awal**	**Fungsi**
jumlah_mahasiswa			
i			
nilai			
total_nilai			
jumlah_lulus			
jumlah_tidak_lulus			
nilai_tertinggi			
nilai_terendah			
rata_rata			
persentase_lulus			

8. Ketentuan Source Code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- menggunakan for;
- menggunakan do-while;
- menggunakan counter;
- menggunakan accumulator;
- menggunakan kondisi untuk menentukan kelulusan;
- memvalidasi jumlah mahasiswa;

- memvalidasi setiap nilai;
- menentukan nilai tertinggi dan terendah;
- menghitung rata-rata dalam bentuk pecahan;
- menghitung persentase kelulusan;
- menggunakan nama variabel yang deskriptif;
- menggunakan indentasi konsisten;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error atau warning penting;
- diuji menggunakan minimal delapan test case;
- disertai trace table;
- dapat dijelaskan oleh mahasiswa.

9. Header Komentar

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 5
Deskripsi   : Program menerima dan mengolah nilai mahasiswa
              menggunakan struktur perulangan.
*/
```

10. Berkas yang Dikumpulkan

- source code: tugas_modul_05_nim.c
- analisis input–process–output;
- daftar variabel;
- pseudocode atau flowchart;
- trace table untuk satu data uji;
- tabel pengujian minimal delapan kasus;
- tangkapan layar kompilasi;
- tangkapan layar hasil eksekusi;
- refleksi;
- pernyataan penggunaan AI.

2. Contoh Test Case Tugas

No.	Jumlah Mahasiswa	Data Nilai	Expected Result Utama
1	0	–	Input jumlah ditolak
2	101	–	Input jumlah ditolak
3	1	60	Rata-rata 60, lulus 1
4	1	0	Rata-rata 0, tidak lulus 1
5	3	50, 60, 70	Rata-rata 60, lulus 2
6	4	100, 100, 100, 100	Rata-rata 100
7	5	80, 55, 70, 90, 60	Rata-rata 71, lulus 4
8	3	-1, 50, 101, 60, 70	Input tidak valid diminta ulang
9	3	0, 0, 0	Tertinggi dan terendah 0
10	3	100, 0, 50	Tertinggi 100, terendah 0

Nilai tidak valid tidak boleh ikut dihitung sebagai data mahasiswa.

3. Simulasi Tabungan Berbunga

Buat program C yang mensimulasikan pertumbuhan saldo tabungan dengan bunga majemuk sederhana per bulan menggunakan for. Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.

Format Input

Tiga baris: saldo awal (double), suku bunga per bulan dalam persen (double), dan jumlah bulan (int).

Batasan

- $1000 \leq$ saldo awal ≤ 1000000000;
- $0 \leq$ suku bunga ≤ 10 (persen per bulan);
- $1 \leq$ jumlah bulan ≤ 360.

Proses

Setiap bulan, saldo bertambah sebesar suku bunga dikalikan saldo saat itu:

```
saldo = saldo + (saldo × suku_bunga ÷ 100)
```

Proses tersebut dilakukan berulang sebanyak jumlah bulan menggunakan for — saldo bulan berikutnya dihitung dari saldo hasil bulan sebelumnya, bukan dari saldo awal.

Contoh 1

Input:

```
1000000
1
3
```

Output:

```
========================================
       SIMULASI TABUNGAN BERBUNGA
========================================
Saldo awal     : Rp1000000.00
Suku bunga     : 1.00% per bulan
Jumlah bulan   : 3
----------------------------------------
Saldo akhir    : Rp1030301.00
========================================
```

Contoh 2

Input:

```
500000
2
2
```

Output:

```
========================================
       SIMULASI TABUNGAN BERBUNGA
========================================
Saldo awal     : Rp500000.00
```

```
Suku bunga     : 2.00% per bulan
Jumlah bulan   : 2
---------------------------------------
Saldo akhir    : Rp520200.00
=======================================
```

Contoh 3

Input:

```
100000
0
5
```

Output:

```
=======================================
       SIMULASI TABUNGAN BERBUNGA
=======================================
Saldo awal     : Rp100000.00
Suku bunga     : 0.00% per bulan
Jumlah bulan   : 5
---------------------------------------
Saldo akhir    : Rp100000.00
=======================================
```

Ketentuan

- gunakan variabel bertipe double untuk saldo, suku bunga, dan for dengan counter bertipe int untuk bulan;
- Contoh 3 menunjukkan kasus suku bunga 0% — saldo akhir harus tetap sama dengan saldo awal, gunakan ini untuk memeriksa loop tidak mengubah saldo secara keliru;
- gunakan %.2f untuk seluruh nilai uang dan suku bunga agar selalu tampil dua angka di belakang koma;
- kedua baris pembatas (garis = dan garis -) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

Simpan sebagai tugas_modul_05_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.

X. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. Accumulator tidak diinisialisasi

Salah:

```
double total;

total += nilai;
```

Benar:

```
double total = 0.0;
```

b. Counter tidak diperbarui

```
int i = 1;

while (i <= 5)
{
    printf("%d\n", i);
}
```

Program mengalami infinite loop.

Perbaikan:

```
i++;
```

c. Update bergerak ke arah yang salah

Salah:

```
for (i = 1; i <= 5; i--)
```

Nilai i semakin kecil dan kondisi terus benar.

Benar:

```
for (i = 1; i <= 5; i++)
```

d. Off-by-one error

```
for (i = 1; i < 5; i++)
```

Hanya menghasilkan empat iterasi. Apabila ingin lima iterasi:

```
for (i = 1; i <= 5; i++)
```

e. Menimpa accumulator

Salah:

```
total = nilai;
```

Nilai sebelumnya hilang. Benar:

```
total += nilai;
```

f. Salah menempatkan input pada while

Salah:

```
scanf("%d", &nilai);

while (nilai != -1)
{
    total += nilai;
}
```

Nilai tidak pernah diperbarui. Benar:

```
scanf("%d", &nilai);

while (nilai != -1)
{
    total += nilai;

    scanf("%d", &nilai);
}
```

g. Sentinel ikut diproses

Salah:

```
while (nilai != -1)
{
    scanf("%d", &nilai);
    total += nilai;
}
```

Nilai -1 dapat ikut ditambahkan sebelum kondisi diperiksa kembali.

Pastikan input dan proses ditempatkan dengan urutan yang benar.

h. Pembagian dengan nol

Salah:

```
rata_rata = total / jumlah_data;
```

tanpa memeriksa apakah jumlah_data lebih besar dari nol. Benar:

```
if (jumlah_data > 0)
{
    rata_rata = total / jumlah_data;
}
```

i. Titik koma setelah for atau while

Salah:

```
for (i = 1; i <= 5; i++);
{
    printf("%d\n", i);
}
```

Perulangan memiliki blok kosong. Blok kurung kurawal dijalankan satu kali setelah perulangan selesai.

Salah:

```
while (i <= 5);
{
    i++;
}
```

Dapat menyebabkan infinite loop.

j. Lupa titik koma pada do-while

Salah:

```
do
{
    printf("%d\n", i);
    i++;
}
while (i <= 5)
```

Benar:

```
do
{
    printf("%d\n", i);
    i++;
}
while (i <= 5);
```

k. Minimum dan maksimum diinisialisasi secara sembarang

Berisiko salah:

```
double nilai_terendah = 0.0;
```

Apabila seluruh nilai positif, nilai terendah dapat tetap 0 meskipun 0 tidak pernah dimasukkan.

Gunakan data pertama:

```
if (i == 1)
{
    nilai_tertinggi = nilai;
    nilai_terendah = nilai;
}
```

l. Mengubah counter di dalam blok

Kurang aman:

```
for (i = 1; i <= 5; i++)
{
    i++;
}
```

Counter diperbarui dua kali sehingga sebagian iterasi terlewati.

2. Standar Penulisan Perulangan

Gunakan nama counter yang jelas

Untuk perulangan sederhana:

```
int i;
```

Untuk konteks tertentu:

```
int nomor_mahasiswa;
```

Gunakan kurung kurawal

```
for (i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Hindari perubahan counter tersembunyi

Hindari:

```
printf("%d\n", i++);
```

Untuk mahasiswa pemula, lebih jelas:

```
printf("%d\n", i);
i++;
```

Pisahkan proses kompleks

Kurang mudah dibaca:

```
total += nilai * bobot / jumlah_data;
```

Lebih jelas:

```
nilai_berbobot = nilai * bobot;
total += nilai_berbobot;
```

Gunakan komentar untuk tujuan, bukan sintaks

Kurang bermanfaat:

```
// Menambah i
i++;
```

Lebih informatif:

```
// Berpindah ke data mahasiswa berikutnya.
i++;
```

XII. Pernyataan Penggunaan AI

Tidak menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[perulangan, error, test case, atau bagian lain]. Hasil tersebut saya
periksa melalui tracing, kompilasi, pengujian kondisi batas, dan
perbandingan dengan algoritma yang telah disusun.
```

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

Knuth, D. E. (1997). The art of computer programming, volume 1: Fundamental algorithms (3rd ed.). Addison-Wesley.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp